Strategy: AI Sales Agent Integration for Services & Bookings

Created: 2026-04-27 **Status:** Proposal (pre-implementation) **Scope:** Per-service AI sales assistant for the FixFlow web + mobile apps

---

Context

The "Create Service" UI (see `c:\dev\sc1.png`) already shows a planned **AI Sales Assistant** toggle with:

- On/off per service
- Tone selection: Friendly / Professional / Urgent
- "Suggest upsells" toggle
- "Enable booking assistance" toggle
- "See How the AI Replies" preview

This document recommends a tooling and architecture path to make that UI functional.

Codebase audit — what's already there

Area	State	Reusable for AI agent?
Service catalog (`shop_services`)	Mature: name, description, price, duration, category, image_url, group rewards, tags	✅ Direct context input
Booking flow (`service_orders`)	Mature: scheduled bookings, time slots, no-show tracking, deposits, cancellations, reschedule	✅ Tool-use targets
Messaging system (`conversations`, `messages`)	Mature: encrypted messages, message types (`text`, `booking_link`, `service_link`, `system`), unread counts, archive/block	✅ AI inserts as `sender_type='shop'` with a flag
Auto-messages (`shop_auto_messages`, `auto_message_sends`)	Mature: schedule + event triggers, target audiences, dedup tracking	✅ Hook for proactive AI outreach
Quick replies (`shop_quick_replies`)	Mature: canned response library	✅ Source of human-tone fingerprint
Web/mobile push (`device_push_tokens`)	Mature (after migration 101 reconciliation)	✅ Notify shops when AI escalates
Customer state (tier, no-show history, RCN balance, group memberships)	Mature	✅ Personalization input
Existing LLM integration	**None**	Greenfield
Existing dependencies relevant to AI	None (`pg`, `express`, `axios`, `stripe`, `thirdweb`, `node-cron`)	Need to add SDK

Bottom line: the messaging and service infrastructure is excellent. The only thing missing is the LLM call itself plus its surrounding conversation/audit/billing scaffolding. This is a small surface area for big leverage.

Recommendation: Anthropic Claude API as the core LLM

Why Claude over alternatives

Tool	Verdict	Reasoning
Anthropic Claude API (Sonnet 4.6 + Haiku 4.5)	✅ **Recommended**	Best-in-class instruction following for tone control (friendly/professional/urgent maps cleanly to system-prompt persona); strong, reliable tool use for booking/upsell actions; native prompt caching cuts cost ~90% for the per-service system prompt; team already uses Claude Code for development so no learning curve; long context (1M tokens) makes shop catalog + customer history fit comfortably without RAG complexity
OpenAI GPT-5/GPT-4o	⚠️ Viable backup	Capable, similar quality, but tool use is slightly less stable for high-stakes actions like booking creation; less generous prompt caching at our scale; nothing currently in the stack uses it
Google Gemini	⚠️ Viable, weaker fit	Strong on multimodal (image input for repair photos = relevant to RepairCoin's original repair use case) but ecosystem split is operational overhead for a small team; revisit if the repair-photo-quote feature ships
Local LLMs (Llama, Mistral, etc.)	❌ No	Operational cost (GPUs, scaling, monitoring, fine-tuning pipelines) far exceeds the SaaS price for our volume; team should focus on product, not ML ops
Aggregator platforms (LangChain, LlamaIndex, Vercel AI SDK)	⚠️ Selectively	LangChain adds abstraction overhead with diminishing returns once you outgrow the templates; **Vercel AI SDK** (`ai` package) is worth using for the *frontend streaming UI* only; backend should call Anthropic SDK directly

Specific model breakdown

Use case	Model	Reason
Sales conversation (booking, upsell, complex Q&A)	**Sonnet 4.6** (`claude-sonnet-4-6`)	Best quality for nuanced sales conversation; tool use stability for booking actions; cached system prompt keeps cost low
Simple FAQ / "what time is your shop open"	**Haiku 4.5** (`claude-haiku-4-5-20251001`)	10x cheaper, fast (sub-second), sufficient for simple lookups
Live-preview rendering ("See How the AI Replies") in shop dashboard	**Haiku 4.5**	Speed matters more than depth here
Tone variant testing / generating shop-specific message templates	**Sonnet 4.6**	One-shot quality matters

(one-shot)		

A simple router can pick the right model per request:

- Detect if the customer is asking a complex multi-turn question (requires history) → Sonnet
- Direct lookup or canned response context → Haiku

Storage layer

Phase 1: no vector database needed.

The per-service context is small enough to fit directly in the prompt:

- Service description (a few hundred tokens)
- Up to 10 sibling services for upsell options (~2K tokens)
- Last 20 messages of the conversation (~3K tokens)
- Customer profile (tier, recent bookings, balance) (~500 tokens)

Total per request: ~6-10K tokens, well within Sonnet's 1M context. **Use prompt caching** on the system prompt + service catalog (which changes rarely) to drop ongoing cost ~90%.

Phase 2 (when needed): pgvector.

If we add cross-shop search (e.g., "find me a repair shop near my location"), use the **pgvector extension on Postgres**. Keeps everything in the existing DB, no new infra. Avoid Pinecone/Weaviate/Chroma until traffic and use case justify the operational burden.

Architecture proposal

New domain: `AIAgentDomain`

Following the existing DDD structure (`backend/src/domains/AIAgentDomain/`):

```
AIAgentDomain/
├── index.ts              # DomainModule registration
├── routes.ts             # Express routes (mounted at /api/ai)
├── controllers/
│   ├── AgentController.ts        # Customer-facing message endpoint
│   ├── PreviewController.ts      # Shop-side "see how AI replies"
│   └── AdminAgentController.ts   # Cost/audit dashboard
├── services/
│   ├── AnthropicClient.ts        # SDK wrapper, retry/backoff
│   ├── AgentOrchestrator.ts      # Main flow: build context → call Claude → handle tool calls
│   ├── ContextBuilder.ts         # Assembles service + customer + conversation context
│   ├── PromptTemplates.ts        # System prompts per tone/use-case
│   ├── ToolHandlers.ts           # Tool implementations (getAvailableSlots, createBooking,
etc.)
│   └── AuditLogger.ts            # Logs every AI request/response for compliance + cost
└── constants.ts          # Token/cost/limit constants
```

Database additions

Two new tables, one column-set on existing:

1. `shop_services` **— add columns** (for the UI toggles):

```sql
ALTER TABLE shop_services
  ADD COLUMN ai_sales_enabled BOOLEAN DEFAULT FALSE,
  ADD COLUMN ai_tone VARCHAR(20) DEFAULT 'professional'  -- 'friendly' | 'professional' | 'urgent'
    CHECK (ai_tone IN ('friendly','professional','urgent')),
  ADD COLUMN ai_suggest_upsells BOOLEAN DEFAULT FALSE,
  ADD COLUMN ai_booking_assistance BOOLEAN DEFAULT FALSE,
  ADD COLUMN ai_custom_instructions TEXT;  -- optional per-service "must mention X" or "never offer Y"
```

2. `ai_agent_messages` (new):

- Mirror of every AI request → response pair
- Fields: `id, conversation_id, service_id, shop_id, customer_address, request_payload (jsonb), response_payload (jsonb), model, input_tokens, output_tokens, cached_input_tokens, cost_usd, tool_calls (jsonb), latency_ms, escalated_to_human, created_at`
- Critical for: cost monitoring, debugging, compliance ("show me what the AI said to my customer"), training data

3. `ai_shop_settings` (new):

- Per-shop overrides (above and beyond per-service)
- Fields: `shop_id, ai_global_enabled, monthly_budget_usd, current_month_spend_usd, escalation_threshold (e.g., always handoff after N AI replies), business_hours_only_ai BOOLEAN, blacklist_keywords TEXT[], created_at, updated_at`
- Lets shop owners cap spend and enforce policies

Conversation flow

1. Customer sends a message in an existing `conversations` thread tied to a `service_id` (or shop-level)
2. Existing `MessageService` inserts the customer message
3. **Hook fires:** if `service.ai_sales_enabled = TRUE` AND `shop.ai_global_enabled = TRUE` AND under budget AND not in escalated state → enqueue AI reply job
4. `AgentOrchestrator`:
   - `ContextBuilder` pulls: service info, last 20 conversation messages, customer profile, sibling services (if upsells enabled), shop hours
   - Builds system prompt from `PromptTemplates[tone]` + custom instructions
   - Calls Claude with tools attached (`getAvailableSlots` , `createBooking` , `getRelatedServices` , `escalateToHuman`)
   - If tool calls returned → execute them → loop back to Claude with results
   - Final response → insert into `messages` with `sender_type='shop'` , `metadata: { generated_by: 'ai_agent', model, tone }`
   - `AuditLogger` writes to `ai_agent_messages`
5. Customer sees the reply in the existing messaging UI (no new client work for the message display itself — it's just another message in the thread, possibly with a small "AI" badge)

Tools (Claude tool use definitions)

```sql
const tools = [
  {
```

```
    name: 'get_available_slots',
    description: 'Look up available appointment time slots for this service in the next N
days',
    input_schema: { type: 'object', properties: { days_ahead: { type: 'integer' } } },
  },
  {
    name: 'create_booking',
    description: 'Create a confirmed booking for the customer at the chosen slot',
    input_schema: { type: 'object', properties: { slot_iso: { type: 'string' }, notes: {
type: 'string' } } },
  },
  {
    name: 'get_related_services',
    description: 'List sibling services from the same shop that could be upsell
opportunities',
    input_schema: { type: 'object', properties: { max: { type: 'integer' } } },
  },
  {
    name: 'escalate_to_human',
    description: 'Stop the AI session and notify the shop owner. Use when customer requests
human help, or AI is uncertain.',
    input_schema: { type: 'object', properties: { reason: { type: 'string' } } },
  },
];
```

The `create_booking` tool **must** require explicit customer confirmation in the AI's prompt — never auto-book without the customer saying "yes." The orchestrator can optionally require a second LLM call to verify intent before executing the tool, OR (simpler) the prompt rules force the model to always ask "shall I confirm this booking?" before calling the tool.

Prompt template skeleton (Friendly tone, one of three)

```
You are a friendly sales assistant for {shop_name}, a {shop_category} business.
You're helping a customer learn about and potentially book "{service_name}".

About this service:
- {service_description}
- Price: ${price_usd}, duration: ~{duration_minutes} minutes
- Category: {category}

About the customer:
- Tier: {customer_tier}
- Recent bookings: {recent_summary}
- Current RCN balance: {rcn_balance}

Style rules:
- Warm, casual tone — feel like a knowledgeable friend
- Match the customer's energy; if they're brief, be brief
- Always disclose you're an AI assistant on the first reply
- {if upsells_enabled}: when natural, mention 1-2 related services from the same shop
- {if booking_assistance}: help them book a slot, but require explicit "yes" before calling
```

```
create_booking
- Never invent prices, hours, or policies not in the context
- If the customer asks something you don't know, escalate_to_human

Custom shop instructions:
{ai_custom_instructions}
```

The Professional and Urgent variants change tone words but keep the same structure.

Phased rollout

Phase 1 — MVP (~3-4 weeks engineering)

Goal: Customer messages a shop about a specific service, AI replies in the existing chat thread, **and helps the customer book the service via an inline action button**. The booking action itself runs through the existing booking UI (Stripe deposit + slot selection) — the AI doesn't directly execute the transaction.

This matches the design's promise on the Create Service page: "*Automatically replies, answers questions,* ***and books customers*** *for this service.*"

MVP capabilities

Capability	In MVP?	Approach
AI replies in chat	✅	Pure context injection, Sonnet for sales / Haiku for FAQ
AI answers questions about the service	✅	Service info + customer profile in context
AI helps customer book (Flavor B)	✅	**AI sends booking action buttons in-chat; customer taps → existing booking UI opens pre-filled**
AI suggests upsells	✅	Mentions related services in conversation; no tool call needed (text-level only)
Live AI reply preview in shop dashboard	✅	`POST /api/ai/preview` endpoint; shows shop how the AI will sound
Per-shop spend cap	✅	Default $20/month, hard auto-throttle to Haiku at 70%
Direct AI tool-call booking (Flavor A — AI auto-creates booking)	❌ Phase 2	Adds confirmation + audit complexity; not necessary if Flavor B works
Full escalation tool with quality scoring	⚠️ Light	MVP detects "talk to a human" intent → notifies shop. Full tool use in Phase 2.

Engineering scope for MVP

- **DB additions:**
  - 4 columns on `shop_services` (the toggles from the UI: `ai_sales_enabled`, `ai_tone`, `ai_suggest_upsells`, `ai_booking_assistance`)
  - Plus `ai_custom_instructions TEXT` column
  - New tables `ai_agent_messages` (audit log), `ai_shop_settings` (per-shop caps + budget)

- **`AIAgentDomain` skeleton** with `AnthropicClient`, `AgentOrchestrator`, `ContextBuilder`, `PromptTemplates`, `AuditLogger`
- **Hook in `MessageService.sendMessage`:** when customer sends → check service flag → enqueue AI reply
- **`BookingSuggestionRenderer` component** (chat UI): renders AI's structured booking suggestion (slot, service, deposit) as an inline tap-to-book card
- **Booking pre-fill:** existing booking UI accepts query params (`?service=X&slot=Y&deposit=Z`) so the AI's button can deep-link the customer into a pre-filled flow
- **Booking-completion event:** when an existing booking finishes (via the existing flow), emit a domain event the AI orchestrator subscribes to → AI generates a confirmation reply in the same chat thread
- **Frontend:** wire the existing toggle UI on the Create Service page to the new DB columns
- **Frontend:** live-preview endpoint (`POST /api/ai/preview` with `service_id` + sample customer question → returns AI reply)
- **AI disclosure badge:** add `metadata: { generated_by: 'ai_agent' }` on AI messages so the UI shows the 🤖 indicator

Out of scope for MVP (deferred to later phases)

- Direct AI tool-call booking (AI calls `create_booking` itself rather than sending a button) — Phase 2
- Full escalation tool with reason capture and SLA tracking — Phase 2
- Quality scoring (thumbs-up / thumbs-down per AI message) — Phase 2
- Customer-initiated "switch to human" button — Phase 2
- Voice / image input — Phase 3
- Cross-shop search (vector embeddings) — Phase 3

Phase 2 — Direct tool use + full escalation (~3-4 weeks after MVP)

Adds the heavier tool-use capabilities for shops that want them, with the safeguards needed for AI-direct booking.

- Implement `create_booking`, `get_available_slots`, `get_related_services`, `escalate_to_human` as Claude tools
- Wire them into the orchestrator's tool-use loop
- **Booking confirmation safeguards:** AI must always ask "shall I confirm this booking?" before tool fires; second-pass intent verification before the actual write; full audit log per tool call
- Push notification to shop when AI escalates
- Customer-side UI to request human handoff (a button in the chat)
- Quality scoring: store thumbs-up/down per AI message; aggregate per-shop and per-tone for tuning
- A/B testing capability: route 10% of conversations to "AI off" baseline to measure conversion lift

Phase 2 mostly upgrades MVP's button-based booking (Flavor B) to direct tool-call booking (Flavor A) for shops that explicitly opt in. Most shops will probably stay on Flavor B forever — it's safer and the customer experience is identical. Phase 2 exists for shops who want zero-friction AI booking despite the higher risk.

Phase 3 — Optimization & expansion (~later)

- **Voice / multimodal:** customer sends a photo of a broken laptop → AI quotes repair cost (very on-brand for FixFlow)
- **Per-shop fine-tuning** (or just better few-shot prompting) using the shop's `quick_replies` and historical `messages` as the human-voice fingerprint
- **Proactive outreach** via existing `auto_messages` infrastructure: AI drafts personalized follow-ups for inactive customers
- **pgvector for service search:** "I need a phone repair near me" → vector search over service descriptions
- **A/B testing infrastructure** to measure conversion lift (AI on vs AI off)

Cost model

Per-conversation rough estimate (5-turn average, with prompt caching):

Model	Input tokens	Output tokens	Cached input	Cost / convo
Sonnet 4.6	3K	600	5K cached	**~$0.018**
Haiku 4.5	3K	600	5K cached	**~$0.0035**

Assumptions:

- System prompt + service catalog (~5K tokens) cached, shared across all customers of that service for 5 minutes
- Conversation history per request: ~3K tokens
- Output ~600 tokens per turn

For a shop with 100 conversations/month, monthly cost is **~$1.80 with Sonnet**, **~$0.35 with Haiku**.

For platform-wide (say 1,000 active shops × 100 convos = 100K conversations), **~$1,800/month with Sonnet**, **~$350/month with Haiku**. Add 30% buffer for tool-use overhead → ~$2,500/month at scale.

Recommended billing/policy:

- Free tier: AI disabled by default
- Standard plan ($500/mo Stripe subscription already in place): includes ~$10 of AI credit per month
- Heavy users: charge passthrough cost + 30% margin via shop's `ai_shop_settings.monthly_budget_usd`
- Hard cap to prevent surprise bills; throttle to Haiku once 70% of budget used

---

Compliance & safety

Concern	Mitigation
Customer not aware they're talking to AI	Disclosure in every first AI reply ("Hi! I'm {Shop}'s AI assistant, here to help..."); UI badge on AI messages; setting in `ai_shop_settings` to enforce always-disclose
AI hallucinates prices, hours, or services	All factual data comes from DB context; system prompt explicitly says "never invent prices, hours, or policies"; every reply auditable in `ai_agent_messages`
AI books appointments customer didn't intend	Tool-use rule: never call `create_booking` without explicit "yes"; double-confirm via a second LLM call OR a UI confirmation step
Prompt injection from customer messages	Sanitize/escape user content; instruct system prompt to ignore meta-instructions in the customer message; rate-limit + monitor for jailbreak patterns
Encrypted messages (migration 097 added support)	AI agent skipped entirely for encrypted threads — those are explicitly customer-to-human only
Shop owner liability for AI-said things	Audit log + clear ToS that shop owner is responsible for AI behavior on their account; allow per-service disable (already in scope)

Customer wants human, AI keeps replying	`escalate_to_human` tool; UI button; shop notification; disable AI for that conversation for 24 hours after escalation
GDPR / data residency	Anthropic supports data residency tiers; messages aren't training data by default; if EU shops onboard, audit Anthropic's data-processing agreement

---

Tradeoffs & open questions

- **Streaming responses:** Anthropic supports streaming; do we want the customer to see token-by-token typing? Adds complexity to the messaging UI but creates the "live agent" feel. **Recommendation:** Phase 2, not MVP.
- **Tone preview before save:** the screenshot's "See How the AI Replies" preview implies live LLM call from shop dashboard. Risk: shop owners burn budget testing. **Recommendation:** Cache previews per (service, tone) combo for 1 hour; show "(cached preview)" indicator.
- **Multilingual support:** Claude handles dozens of languages out of the box. Don't engineer anything specific; it'll Just Work for Filipino/Spanish/etc. shops. The system prompt should say "respond in the customer's language".
- **Shop owner training the AI:** Phase 3 fine-tuning is overkill. Instead, let shop owners write `ai_custom_instructions` per service ("always mention our 30-day warranty", "never offer same-day service on Sundays"). Cheap, effective, transparent.
- **Rate limiting:** customer who sends 100 messages in a minute should not run up the shop's AI bill. Enforce: max 5 AI replies per customer per hour per shop. After that, fall back to "shop will reply soon" canned response.
- **Tool-use stability:** Claude's tool use is good but not perfect. Phase 2 should have a feature flag to A/B between tool-use mode (AI books directly) and "draft mode" (AI writes a suggested message + booking link, shop owner one-click sends).
- **Backup model:** if Anthropic has an outage, the system should gracefully degrade to "Sorry, I'll get a human to reply soon" rather than 500 errors. Build this from day one — don't make AI a critical-path dependency for messaging itself.

What to do next (concrete first steps)

1. **Get an Anthropic API key.** Free trial covers proof-of-concept. Move to a paid tier with prompt caching enabled.
2. **Confirm the four UI columns** on the screenshot match what's planned. If yes, write migration 107 (or whichever version is next):

   ```
   ALTER TABLE shop_services ADD COLUMN ai_sales_enabled ...
   ```

3. **Spike: build a single endpoint** `POST /api/ai/preview` that takes a service ID + sample question, calls Claude with the tone-templated system prompt, returns the reply. This proves the round-trip works end-to-end before committing to architecture.
4. **Decide MVP scope cutoff** — Phase 1 as described, or include tool use? My recommendation: Phase 1 first, ship and learn from real customer messages for 2 weeks, then Phase 2.
5. **Budget alignment** — finalize per-shop monthly cap and how it fits into the $500/mo Stripe subscription tier (is AI included? add-on? metered?).

Appendix — Why not LangChain / LlamaIndex / agentic frameworks

These frameworks shine when you need to:

- Coordinate dozens of tools across many models
- Build complex multi-step planning (research → analyze → write → review)
- Plug in many vector stores / data sources

For a per-service sales assistant with 4 tools and a single model, they add abstraction overhead with no benefit. Use the Anthropic SDK directly:

```
import Anthropic from '@anthropic-ai/sdk';

const client = new Anthropic();
const response = await client.messages.create({
  model: 'claude-sonnet-4-6',
  max_tokens: 1024,
  system: [
    { type: 'text', text: SYSTEM_PROMPT, cache_control: { type: 'ephemeral' } },
    { type: 'text', text: SERVICE_CATALOG_CONTEXT, cache_control: { type: 'ephemeral' } },
  ],
  messages: conversationHistory,
  tools: TOOL_DEFINITIONS,
});
```

That's the entire integration. ~50 lines of TypeScript including error handling. Don't pile abstractions on it.

If, in Phase 3, the team wants to build agentic flows (e.g., AI plans a multi-shop booking trip), revisit. Until then, the SDK is enough.

TL;DR

- **Stack:** Anthropic Claude API (Sonnet 4.6 for sales conversations, Haiku 4.5 for simple lookups). Direct SDK use, no LangChain.
- **Storage:** Postgres + prompt caching now; pgvector later if cross-shop search ships. Skip dedicated vector DBs.
- **Domain:** new `AIAgentDomain` mirroring existing DDD pattern; new tables for audit and per-shop settings; 4 columns on `shop_services` to back the UI toggles.
- **Phase 1 (2-3 weeks):** text-only AI replies hooked into existing messaging. No tool use. Cost cap per shop.
- **Phase 2 (4-6 weeks):** tool use for booking + upsell + escalation.
- **Phase 3 (later):** voice/photo, fine-tuning, proactive outreach.
- **Cost:** ~$0.02 per conversation with caching; affordable to bundle into the existing $500/mo subscription tier.
- **Safety:** mandatory disclosure, no auto-booking without confirmation, human escalation tool, audit log, per-shop budget cap.